Matplotlib & Seaborn Fundamentals

Suman 24 Feb, 2026 At 8:00 PM Pre-Reads Module-1 Recommended Resource

Associated Lectures (1)

Description Discussions

Session 5.1: Pre-read Material

Matplotlib & Seaborn Fundamentals

Program: Vishlesan i-Hub IIT Patna x Masai School -- AIM (AI & Machine Learning) **Session ID:** 5.1 I **Week:** 5 **Prerequisites:** Session 4.3 (Data Quality: Principles & Industry Horror Stories) **Session Duration:** 105 minutes **Preparation Time:** 20-30 minutes

What This Session Covers

This session marks the beginning of Block 3 (Visualization & Math Foundations) and teaches you to create visual representations of data -- the skill that turns analysis into influence. The session covers seven topics:

- **Why Visualization?** -- COVID-19 misleading charts and Anscombe's Quartet demonstrate why visualization is both powerful and dangerous
- **Matplotlib Architecture** -- Understanding the Figure/Axes/Artist hierarchy and creating your first line plot
- **Core Plot Types** -- Mastering line, bar, scatter, and histogram plots with an Indian e-commerce dataset
- **Seaborn Statistical Plots** -- Creating countplots, boxplots, heatmaps, and pairplots for rapid exploratory data analysis
- **Customization & Styling** -- Adding titles, labels, legends, color palettes, and professional themes
- **Subplots, Annotations & Export Quality** -- Building multi-panel figures, highlighting key data points, and exporting at 300 DPI
- **Mini-Project: E-commerce Sales Dashboard** -- Assembling 4-5 publication-quality plots on a single canvas

By the end of this session, you will be able to create publication-quality static visualizations, choose appropriate chart types based on the data and question being asked, and understand that HOW you present data is just as important as WHAT the data contains.

Prerequisites Checklist

Before attending this session, make sure you are comfortable with:

- [ ] Loading DataFrames from CSV files with `pd.read_csv()` and exploring with `.head()`, `.info()`, `.describe()` (Session 4.1)
- [ ] Filtering rows with boolean conditions: `df[df['column'] > value]` (Session 4.1)
- [ ] Handling missing values with `.isnull()`, `.dropna()`, `.fillna()` (Session 4.2)
- [ ] Aggregating data with `.groupby()` and aggregate functions like `.mean()`, `.sum()`, `.count()` (Session 4.2)
- [ ] Understanding the six data quality dimensions: accuracy, completeness, consistency, timeliness, validity, uniqueness (Session 4.3)
- [ ] Knowing when to trust a dataset and when to audit it before analysis (Session 4.3)
- [ ] All Python fundamentals (variables, control flow, data structures, functions) from Weeks 1-2
- [ ] All NumPy fundamentals (array creation, indexing, operations) from Week 3

If any of these feel unclear, review your Session 4.1-4.3 materials before class.

Setup Requirements

Python Version

Python **3.12.x** (the version you have been using since Week 1).

Required Packages

You will need **Matplotlib** and **Seaborn** for this session. Install them if you have not already:

```
pip install matplotlib seaborn
```

If you are using Anaconda, use:

```
conda install matplotlib seaborn
```

Verification Code

Run this in a Jupyter Notebook cell or a Python script before class:

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

print(f"Matplotlib version: {plt.matplotlib.__version__}")
```

```
print(f"Seaborn version: {sns.__version__}")
print(f"Pandas version: {pd.__version__}")
print(f"NumPy version: {np.__version__}")
print("Ready for Session 5.1!")
```

You should see version numbers printed (Matplotlib 3.x, Seaborn 0.x, Pandas 2.x, NumPy 1.x or 2.x) and the message "Ready for Session 5.1!" with no errors.

If you see an `ImportError` or `ModuleNotFoundError`, install the missing package using pip or conda and try again.

Business Story: COVID-19 Misleading Charts

Before you learn any plotting techniques, you need to understand what happens when visualization is used carelessly -- or deliberately deceptively.

When Government Health Charts Lied to the Public

During the COVID-19 pandemic, public health decisions depended on data visualization. Charts showing case counts, hospitalizations, and deaths shaped policy, behavior, and public opinion for millions of people. Government agencies and media outlets published these visualizations daily -- and some of them were designed to mislead.

The Georgia Bar Chart Scandal (May 2020): The Georgia Department of Public Health published a bar chart showing COVID-19 case counts by county over time. The chart appeared to show cases declining -- exactly what everyone wanted to see. But when data journalists examined the chart closely, they discovered the dates on the x-axis were **not in chronological order**. The state health department had deliberately rearranged the dates to create a visual pattern of decline that did not exist in reality. When this was discovered, it became a national scandal.

The data itself was accurate -- the numbers were real case counts from real counties. But the CHART lied by presenting those accurate numbers in a deliberately deceptive order.

Truncated Y-Axes: Media outlets published charts where the y-axis started at a non-zero value to exaggerate small changes or hide large ones. Fox News aired a bar chart where y-axis jumps made a 0.2% change visually appear to be a 500% change. Your eye measures the HEIGHT of the bars relative to the frame, not the numeric labels on the axis.

Cherry-Picked Timeframes: Charts showed declining case counts by omitting the most recent weeks of data, hiding alarming upward trends. By choosing where to start and stop the timeline, you can make any trend appear to rise or fall.

Misleading Comparisons: Charts compared raw case counts between states with wildly different populations without normalizing per capita, making small states with high per-capita infection rates appear safe and large states with low per-capita rates appear dangerous.

Why This Matters for You

These examples demonstrate that even ACCURATE, HIGH-QUALITY data can be turned into misinformation through poor -- or intentionally deceptive -- visualization choices. You learned in Session 4.3 that bad data leads to bad decisions. Now you must understand the second path from good data to bad decisions: BAD VISUALIZATION.

Even if your data passes all six quality dimensions -- accurate, complete, consistent, timely, valid, and unique -- if you put it in a misleading chart, the audience will draw the wrong conclusion. **Visualization is not neutral. It is an argument.** And the chart-maker bears responsibility for presenting data honestly.

Think About:

- If you received a chart from a colleague showing declining sales, what would you check before accepting the conclusion?
- Can you think of other situations where someone used a chart to make a claim that "felt wrong" even though the data was real?
- What ethical obligation does a data professional have when creating visualizations that will influence business decisions?

Concepts to Preview

Read through these previews carefully. You do not need to memorize every detail -- the goal is to arrive in class with a general sense of what each concept is, so the discussions and demonstrations feel familiar rather than foreign.

Why Visualize? Anscombe's Quartet

In 1973, statistician Francis Anscombe created four datasets that are now the most famous example in data visualization history. Each dataset has:

- Same mean of x: 9.0
- Same mean of y: 7.5
- Same variance of x: 11.0
- Same variance of y: 4.1
- Same correlation: 0.816
- Same linear regression line: y = 3 + 0.5x

If you only looked at summary statistics, you would conclude these four datasets are identical. But when you plot them:

- **Dataset I:** Clean linear relationship (exactly what the statistics predict)
- **Dataset II:** Curved quadratic relationship (the straight line is the WRONG model)
- **Dataset III:** Perfect linear relationship EXCEPT one extreme outlier pulling the line
- **Dataset IV:** All points stacked at the same x-value except one extreme point defining the entire slope

This is THE classic argument for always visualizing your data. Summary statistics can hide patterns, outliers, and non-linear relationships. Visualization reveals the truth.

Think About:

- Have you ever computed an average and assumed it told the whole story? What could the average be hiding?
- If you removed the extreme outlier in Dataset III, how would the regression line change? What if you removed it in Dataset IV?

Matplotlib Architecture: Figure, Axes, and Artist

Matplotlib can be confusing if you do not understand its mental model. Every plot in Matplotlib consists of three layers:

Figure: The entire canvas or window. This is the container that holds everything.

Axes: A single plot area within a Figure. This is where data is actually drawn. (Important: "Axes" in Matplotlib means "plot area", NOT "axis" -- the x-axis and y-axis are separate objects.)

Artist: Any element drawn on an Axes -- lines, bars, points, text, tick marks, legends. Everything you see is an Artist.

The house analogy: The Figure is the entire plot of land. An Axes is a room in the house built on that land. Artists are the furniture, decorations, and objects inside the room.

There are two ways to create plots in Matplotlib:

- **pyplot interface (stateful, simple):** `plt.plot(x, y)` -- Matplotlib automatically creates a Figure and Axes behind the scenes. This is convenient for quick exploration but limits control.
- **Object-oriented interface (explicit, professional):** `fig, ax = plt.subplots()` followed by `ax.plot(x, y)` -- You explicitly create the Figure and Axes objects and have full control over every element.

In this session, you will learn the object-oriented interface because it is the industry standard for production code and multi-panel figures.

Think About:

- Why might explicit control over Figure and Axes objects be important when creating a dashboard with multiple plots?
- What advantages might the simple pyplot interface have for quick exploratory analysis?

Four Core Plot Types

Every chart type answers a specific analytical question. The key skill is choosing the right plot for the question you are asking:

Plot Type	When to Use	Question It Answers	Visual Element
Line Chart	Time series, sequential data	How does Y change over time or sequence?	Connected line showing trend
Bar Chart	Comparing categories	Which category has the highest/lowest value?	Bars of different heights
Scatter Plot	Relationship between two numeric variables	Is there a correlation between X and Y?	Individual points showing association

Plot Type	When to Use	Question It Answers	Visual Element
Histogram	Distribution of a single numeric variable	What is the shape and spread of this variable?	Bins showing frequency

Common mistakes beginners make:

- Using a line chart for categorical data (categories do not have a natural sequence)
- Using a bar chart for continuous numeric data (bar charts are for discrete categories)
- Using a pie chart when a bar chart would be clearer (pie charts are notoriously hard to read accurately)

In this session, you will create all four core types using an Indian e-commerce dataset with orders, product categories, cities, quantities, prices, and customer ratings.

Think About:

- If you wanted to show "revenue per month over a year", which chart type would you choose? Why?
- If you wanted to show "total revenue by product category", which chart type would you choose? Why?
- If you wanted to show "the relationship between unit price and customer rating", which chart type would you choose? Why?

Seaborn vs Matplotlib

Seaborn is a high-level statistical visualization library built on top of Matplotlib. Think of Matplotlib as the engine and Seaborn as the luxury car built on that engine.

Why Seaborn exists:

- Matplotlib gives you complete control but requires many lines of code to create complex statistical plots
- Seaborn provides simple, one-line functions for common statistical visualizations
- Seaborn has beautiful default color palettes and themes that require zero configuration

When to use each:

- Use **Matplotlib** when you need precise control over every element of the plot
- Use **Seaborn** for rapid exploratory data analysis and statistical plots
- Use **both together** -- Seaborn for the main plot, Matplotlib for fine-tuning labels, titles, and export

Key Seaborn plot types you will learn:

- **countplot:** Bar chart showing frequency of categorical values (like a histogram for categories)
- **boxplot:** Shows median, quartiles, and outliers for a numeric variable
- **heatmap:** Color-coded matrix showing correlation between variables
- **pairplot:** Grid of scatter plots showing relationships between ALL numeric columns

Think About:

- If you wanted to quickly explore correlations between 10 numeric variables in a dataset, would you manually create 45 scatter plots or use a tool that automates it?

Chart Customization Basics

A chart without labels, titles, and legends is like a research paper without citations -- technically functional but unprofessional and hard to trust.

Essential elements every chart needs:

- **Title:** What does this chart show? (e.g., "Monthly Revenue Trend, Jan-Dec 2024")
- **Axis labels:** What are the units? (e.g., "Revenue (INR Thousands)" for y-axis, "Month" for x-axis)
- **Legend:** If multiple series are plotted, which color/line represents what?
- **Appropriate colors:** Use color palettes that are colorblind-friendly and print well in grayscale

Professional touches:

- **Themes:** Seaborn provides built-in themes like `darkgrid`, `whitegrid`, `white`, `dark`, `ticks` that apply consistent styling
- **Color palettes:** Seaborn provides palettes like `deep`, `muted`, `pastel`, `bright`, `dark`, `colorblind` for different use cases
- **DPI (Dots Per Inch):** 72 DPI is fine for screens, 300 DPI is required for publication-quality prints

The resume analogy: Would you submit a resume with no contact information, inconsistent fonts, and spelling errors? No -- because presentation matters. The same is true for charts. A chart with missing labels, garish colors, and low resolution tells the audience "I did not care enough to do this right."

Think About:

- Have you ever seen a chart where you could not figure out what the axes represented? How did that affect your trust in the data?
- Why might exporting at 300 DPI matter if you are including charts in a business report or research paper?

Key Vocabulary

These terms will appear throughout the session. Familiarize yourself with them so they are not completely new when you hear them in class.

Term	Definition
Figure	The entire canvas/window in Matplotlib -- the top-level container for all plot elements
Axes	A single plot area within a Figure (NOT the same as "axis" -- this is where data is drawn)
Axis	The x-axis or y-axis line with tick marks and labels (a component of an Axes object)

Term	Definition
Artist	Any element drawn on an Axes -- line, bar, text, tick mark, legend, etc.
pyplot	Matplotlib's simple stateful interface (`plt.plot()`) that auto-creates Figures and Axes
Object-Oriented (OO)	Matplotlib's professional interface (`fig, ax = plt.subplots()`) with explicit control
DPI	Dots Per Inch -- controls export resolution (72 for screen, 300 for publication)
Seaborn	High-level statistical visualization library built on top of Matplotlib
Anscombe's Quartet	Four datasets with identical statistics but completely different visual patterns
Line Chart	Plot type showing trend over time or sequence with connected lines
Bar Chart	Plot type comparing categorical values with rectangular bars of different heights
Scatter Plot	Plot type showing relationship between two numeric variables as individual points
Histogram	Plot type showing distribution of a single numeric variable using frequency bins
Boxplot	Chart showing median, quartiles (Q1, Q3), and outliers for a numeric variable
Heatmap	Color-coded matrix showing correlation or relationships between variables
Pairplot	Grid of scatter plots showing relationships between all numeric columns in a dataset
Countplot	Seaborn's bar chart for categorical frequency (like a histogram for categories)
Subplot	One of multiple Axes arranged in a grid on a single Figure
Annotation	Text or arrow added to a plot to highlight a specific data point or region
Legend	Box explaining what each color/line/marker represents in a multi-series plot
Color Palette	Coordinated set of colors applied consistently across plots
Theme	Pre-configured style settings (grid, background, font) applied to all plots

Things to Ponder

Come to class having thought about these questions. You do not need written answers, but engaging with them beforehand will help you follow the session more actively.

About trust: When you see a chart in a news article, business presentation, or research paper, do you instinctively trust it -- or do you check the axes, timeframe, and data source first? What would make you distrust a chart even if the underlying data was accurate?

About defaults: Excel, Google Sheets, and many tools auto-generate charts with default settings. Can you think of situations where accepting the default chart design could produce misleading visualizations? (Hint: think about y-axis scales, chart types, and color choices.)

About patterns: Anscombe's Quartet shows that summary statistics can hide completely different patterns. Can you think of a real-world example where an average or correlation was reported without visualization, and important details were missed?

About responsibility: If you create a chart that technically shows accurate data but is formatted in a way that leads executives to make a wrong decision, who is responsible -- you for creating the chart, or the executives for not questioning it?

About choice: If you have a dataset with monthly sales data and you want to convince management to invest more in marketing, which chart type would make the strongest argument -- a line chart showing smooth trends, a bar chart emphasizing monthly peaks, or a scatter plot showing variability? Does choosing the most persuasive visualization cross the line into manipulation?

Optional Deep Dive

For ambitious students who want to explore beyond the session content:

Articles & Tutorials

- Matplotlib Official Tutorial -- Comprehensive guide to all Matplotlib features
- Seaborn Official Tutorial -- Statistical visualization examples with code
- "How to Lie with Charts" by Alberto Cairo -- Classic text on deceptive visualization techniques (beware: some learn this to avoid mistakes, others to exploit them)
- Data Visualization Catalogue -- Browse 60+ chart types with use cases and examples

Advanced Concepts (Week 6+)

- **Interactive Visualizations:** Plotly, Bokeh, Altair for web-based dashboards
- **Geographic Visualizations:** Folium, GeoPandas for maps
- **3D Plots:** Matplotlib's `mplot3d` toolkit (rarely needed, often misused)
- **Animation:** Matplotlib's `FuncAnimation` for time-series animations

Research Papers

- Anscombe, F.J. (1973). "Graphs in Statistical Analysis". *American Statistician* 27(1): 17-21. -- The original Anscombe's Quartet paper

---

Pre-Class Checklist

Before you walk into class, make sure you can check off every item:

- [ ] I have Python 3.12.x installed with Jupyter Notebook (or JupyterLab)
- [ ] I can `import matplotlib.pyplot as plt` and `import seaborn as sns` without errors
- [ ] I have read the COVID-19 misleading charts business story and understand why visualization can deceive
- [ ] I know what Anscombe's Quartet demonstrates and why summary statistics alone are insufficient
- [ ] I understand the Figure/Axes/Artist hierarchy in Matplotlib
- [ ] I can name the four core plot types and explain when to use each one
- [ ] I know the difference between the pyplot interface (`plt.plot()`) and the object-oriented interface (`fig, ax = plt.subplots()`)
- [ ] I understand what DPI means and why 300 DPI is required for publication-quality exports
- [ ] I remember how to load a CSV with `pd.read_csv()` and filter rows with boolean conditions (Session 4.1)
- [ ] I remember how to use `.groupby()` to aggregate data (Session 4.2)
- [ ] I have reviewed the vocabulary table above

Looking Ahead

After this session, you will be able to:

- Explain why data visualization is essential for analysis and identify common deception techniques
- Create line, bar, scatter, and histogram plots using Matplotlib's object-oriented interface
- Use Seaborn to rapidly create statistical plots (countplot, boxplot, heatmap, pairplot)
- Customize charts with titles, axis labels, legends, color palettes, and themes
- Build multi-panel figures using `plt.subplots(nrows, ncols)` and add annotations
- Export visualizations at publication quality (300 DPI PNG, SVG, PDF)
- Choose appropriate chart types based on the data and the analytical question being asked

The mini-project: During the session, you will build an **E-commerce Sales Dashboard** using a custom Indian e-commerce dataset. You will create 4-5 plots on a single Figure:

> Monthly revenue line chart with an annotated peak month
>
> Horizontal bar chart of total revenue by product category
>
> Scatter plot of unit price vs customer rating, colored by category
>
> Histogram of order value distribution with a mean line
>
> Correlation heatmap of numeric columns

You will apply consistent styling, add professional labels and titles, and export the dashboard at 300 DPI -- ready to include in a business report.

The bridge to Week 5: Session 5.1 teaches you STATIC visualizations -- charts that communicate insights clearly and honestly. Session 5.2 will teach **Data Storytelling & Interactive Dashboards** -- how to structure a visual narrative and build web-based dashboards with Plotly Dash. Session 5.3 covers **Linear Algebra Fundamentals** -- the mathematical foundation for understanding machine learning. By the

end of Week 5, you will have completed the toolkit for communicating insights (visualization) and begun the mathematical foundations for building models.

The 80/20 reality revisited: You learned in Session 4.3 that data scientists spend ~80% of their time on data preparation and ~20% on modeling. Within that 80%, a significant portion is spent on **exploratory data analysis (EDA)** -- visualizing data to understand patterns, detect outliers, and validate assumptions before any analysis begins. The visualization skills you build today are not just for presentations -- they are for YOUR understanding. Anscombe's Quartet proved this definitively: you cannot trust statistics alone. You must visualize.

As Edward Tufte, the pioneer of data visualization, wrote: *"Graphical excellence is that which gives to the viewer the greatest number of ideas in the shortest time with the least ink in the smallest space."* Your goal is not to make pretty charts. Your goal is to reveal truth efficiently and honestly.

See you in class!

Vishlesan i-Hub IIT Patna x Masai School